School of Computer Science and Engineering

BCSE306L - Artificial Intelligence

Course Project Review - 3

LabelGuard

A Personalised Food-Label Scanner for Indian Households using LLM Vision and a Deterministic Rules Engine

PROJECT REPORT

Submitted by

Name	Register No.
Harsh Kulkarni	24BCT0137
B S Srivallabha	24BCE0457
Ayush Konar	24BDS0107
Pratham Paudel	24BCE2940

Faculty	Dr. D. Anuradha
Slot	C2+TC2
Semester	Fall Semester 2026-27
Date	November 2026

Abstract

Packaged-food labels in India are hard to act on: sugar is split across many names (jaggery, invert syrup, maltodextrin), additives appear only as INS codes, front-of-pack words such as *healthy*, *natural* or *made with real fruit* are rarely checked against the ingredient list, and one household often has very different needs (a Jain diet, a pre-diabetic adult, a lactose-intolerant child). General-purpose chatbots can read a label but give a different answer each time, forget the family between conversations and may invent products.

LabelGuard is a web application that photographs or looks up a packaged food and produces a personalised, repeatable verdict for every family member. Its design principle is: *the LLM handles perception and interpretation; deterministic Python rules and real databases handle anything that must be consistent, verifiable or cheap.* A Gemini vision model reads one to three photos of a pack (front and back) into structured JSON in a single call. A rules engine with curated Indian lookup tables (54 sugar aliases, 49 INS additives, 22 allergen groups, 34 Jain/vegetarian/vegan/halal rules) then flags allergens, diet conflicts and medical-condition risks with evidence, checks label claims against FSSAI thresholds, compares front-of-pack promises with the back of the pack, and computes a transparent 0-100 health score. Better alternatives come from Open Food Facts and a 73-product Indian table, scored with the same formula. A tracker, insights page and weekly family report close the loop.

All safety-relevant outputs are deterministic (identical input gives identical output, verified by 46 automated tests); the LLM is used for only three core tasks plus two optional ones, every output is cached, and an automatic multi-model fallback keeps scans working when a model is overloaded. A typical new scan uses about 2,100 input and 450 output tokens and completes in roughly 5 seconds, while repeat scans, barcode lookups and every analysis module cost zero tokens.

Keywords: food labelling, large language models, multimodal extraction, rules engine, personalised nutrition, FSSAI, front-of-pack claims, Open Food Facts.

1. Introduction

1.1 Background

India's packaged-food market has grown rapidly, and with it the share of daily sugar, salt and fat that comes from labelled products. FSSAI regulations require an ingredient list, a nutrition table and allergen declarations, and they define thresholds for claims such as *low sugar* or *high fibre*. In practice, consumers face three gaps: the information is technical (INS codes, per-100 g values, hidden sugar names), it is not personal (a label cannot know that one family member is Jain and another is diabetic), and marketing on the front of the pack frequently contradicts the back.

1.2 Problem statement

Design and build a system that, from a photo (or barcode, or typed label) of a packaged food, can:

- accurately extract ingredients, nutrition values, serving size, claims and front-of-pack imagery;
- evaluate the product separately for each household member's allergies, diet (Jain, vegetarian, vegan, halal, eggetarian), medical conditions and daily limits, giving a SAFE / CAUTION / AVOID verdict with evidence;
- produce the **same** score and verdict every time for the same label, with a documented formula;
- detect hidden sugars, explain additives, check claims and expose front-versus-back contradictions;
- suggest real, safer alternatives and track intake over a week; and
- keep LLM token usage low and remain usable when the LLM or the internet is unavailable.

1.3 Why not a general chatbot?

Requirement	Plain chatbot	LabelGuard
Consistent score	Varies between runs	Deterministic formula; identical result for identical input
Family memory	Re-explain every time	Profile saved once, applied to every member on every scan
Alternatives	May invent products	Real products from a database, same scoring
Indian context	Generic	Jain/halal rules, Indian names (kaju, besan), INS codes, FSSAI limits
History	None	Intake tracker, insights, weekly family report
Cost	Long prompt per question	One vision call per new product; cache makes repeats free

1.4 Objectives and scope

The project targets a demonstrable, single-household web application for Indian users. Medical diagnosis, verified brand databases and multi-user accounts are out of scope; every screen carries the disclaimer that the output is informational and not medical advice.

2. Methodology

2.1 System architecture

LabelGuard is a three-tier application (Figure 1). A React single-page app talks to a FastAPI backend over JSON. The backend contains four engines: an LLM layer that wraps Google Gemini, a deterministic rules engine, an alternatives engine and a tracker/report module. Persistent state lives in a single SQLite file; reference knowledge lives in versioned CSV/JSON tables; product data comes from Open Food Facts with a local Indian fallback table.

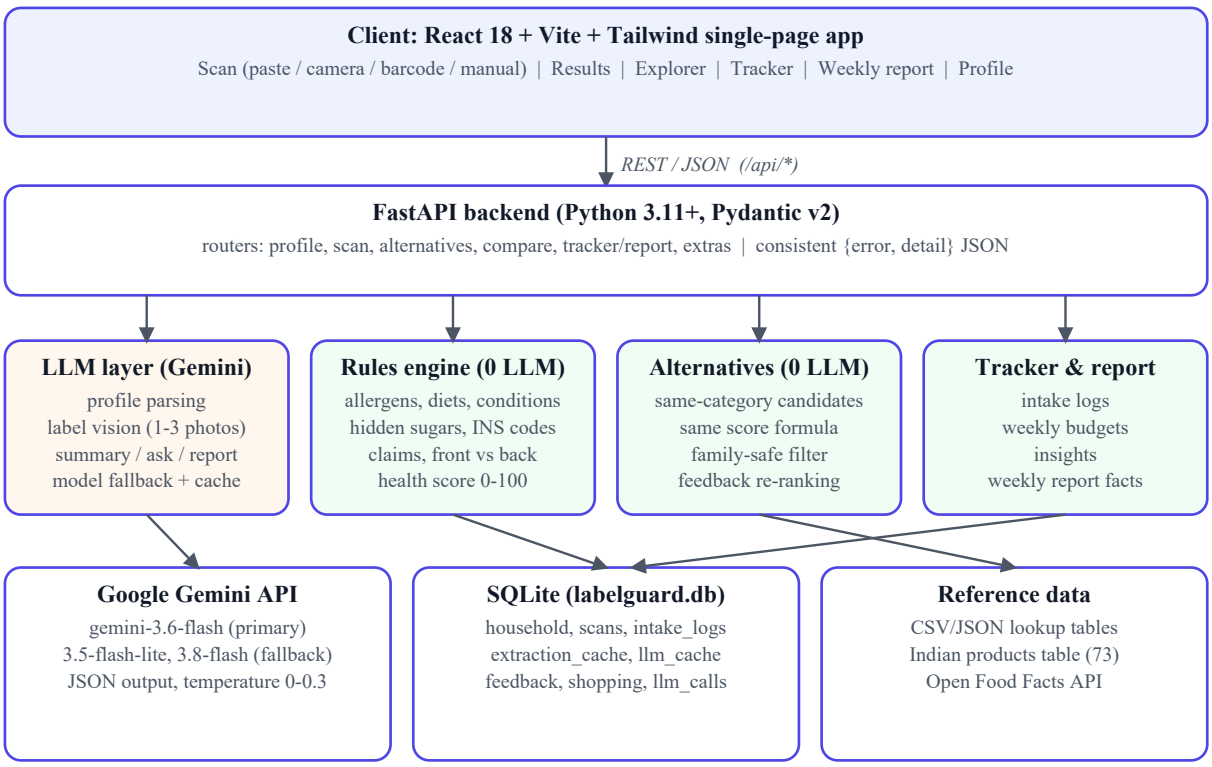


Figure 1: LabelGuard architecture. Orange components call the LLM; green components are pure Python.

2.2 Division of work between the LLM and the rules

LLM task (Gemini)	When it runs	Fallback if unavailable
1. Profile parsing: free text to structured household JSON	Once per profile edit (cached)	Keyword parser
2. Label extraction: 1-3 photos to JSON (incl. front-of-pack imagery)	Once per new image set (cached by hash)	Barcode / manual entry / demo labels
3. Personalised 2-3 line summary (EN / HI / TA, ELI12 option)	Per scan + language (cached)	Template sentences
4. Optional: answer a question about a product	On demand (cached)	Answer built from rule output
5. Optional: weekly family report write-up	On demand (cached)	Template report

Everything else - flags, scores, claim checks, front/back checks, alternatives, tracking, comparison, insights and allergy cards - is deterministic Python with zero LLM calls. The LLM never decides a verdict: summaries and answers receive the rule results as facts and are instructed not to change them.

2.3 Modules

Module	Description
Family profile	Free text parsed into members (name, relation, age, allergies, diet, conditions, daily sugar/sodium/sat-fat limits, ingredients to avoid); editable form; saved in SQLite.
Scan	Three inputs: paste/camera photos (up to 3 of one pack), barcode lookup (Open Food Facts, 0 tokens) and manual entry. Images resized to 1024 px; results cached by image hash and barcode.
Result analysis	Score ring, per-member verdicts with evidence, hidden sugars, additive cards, claim check, front-vs-back check, traffic-light label, per-serving table, summary with text-to-speech, Ask box, share and shopping list.
Alternatives	Top 3 safer, higher-scoring products in the same category with a reason and weekly 'swap impact'; thumbs up/down feedback re-ranks future results.
Compare	2-4 products side by side; winner = safest verdict for the selected member, then highest score.
Explorer & decoder	Browse 73 Indian products with family verdicts; decode any INS code or ingredient against every member.
Tracker, insights, report	Log intake per member; weekly budgets and daily charts; aggregate patterns; LLM-written weekly family report over computed facts.
Allergy cards	Printable per-member card listing allergens with Indian names, diet rules and conditions.

2.4 Algorithms

Algorithm 1 - Label extraction with caching and model fallback

- Compute SHA-256 over the uploaded image(s); if the hash (or a decoded barcode) is in *extraction_cache*, reuse it (0 tokens).
- Otherwise validate and resize each image to at most 1024 px and send all images with the extraction prompt in one Gemini call (JSON output, temperature 0, 'use null, never guess', salt-to-sodium rule sodium_mg = salt_g x 400).
- If the model returns 429/5xx or times out (20 s), retry on the next model in the fallback list; a failed model is skipped for 5 minutes.
- Validate with Pydantic: coerce numbers, derive per-100 g values from per-serving values when only those are printed, and cache the result.

Algorithm 2 - Rules engine

Ingredients are normalised and matched with word-boundary regular expressions that tolerate plurals (so *maltodextrin* never matches *dextrin* and *buckwheat* never matches *wheat*). INS/E-numbers are extracted from forms such as *E471*, *INS 330* and *Emulsifiers (322, 471)*. Exclusion lists prevent false positives (coconut milk is not dairy, fox nuts are not tree nuts). For each member the engine emits flags from allergens (AVOID if listed, CAUTION if 'may contain' or an unspecified parent group such as 'nuts'), diets (AVOID for forbidden items such as onion for Jain; CAUTION 'source unclear' for ambiguous items such as E471 or natural flavours), conditions (e.g. sugar > 22.5 g/100 g is AVOID for diabetes; sodium > 600 mg is AVOID for hypertension), personal limits (one serving using ≥ 40% of a daily sugar limit) and child rules (Southampton colours, caffeine, sweeteners). The member's status is the worst flag.

Algorithm 3 - Health score (0-100, deterministic)

Component	Rule	Max
Sugar	Solids 0.875 pts per g/100 g; drinks 3.5 pts per g/100 ml	-35
Sodium	0.025 pts per mg/100 g	-25
Saturated fat	Solids 2 pts per g; drinks 4 pts per g	-20
Energy density	Solids (kcal - 250)/30; drinks (kcal - 40)/8	-10
Additives	High concern 5, medium 2, low 0.5 each	-15
Processing level	Minimal 0, processed (or unknown) 5, ultra-processed 10	-10
Fibre / protein bonus	+1.6 pts per g fibre; +0.7 pts per g protein	+8 / +7

Score = clamp(100 - penalties + bonuses, 0, 100); each component is rounded to one decimal before summing. Grades: 70-100 *Good choice* (green), 50-69 *Okay occasionally* (amber), 0-49 *Limit this* (red). Thresholds follow UK FSA traffic-light levels, WHO sugar and sodium guidance and a NOVA-style processing estimate.

Algorithm 4 - Claim checker and front-vs-back check

Claims are compared with FSSAI thresholds: *sugar free* ≤ 0.5 g/100 g, *low sugar* ≤ 5 g (2.5 g for drinks), *low sodium* ≤ 120 mg, *high fibre* ≥ 6 g, *high protein* ≥ 20% of energy; *no added sugar* fails if any sugar alias is present; *multigrain/whole wheat* fails if the first ingredient is refined flour; vague words (*healthy*, *natural*) fail when sugar, sodium or saturated fat is high or the score is below 50. For the front of the pack, each pictured food is located in the ingredient list: it is misleading if absent, below 5%, or below 15% when the product is named after it. Messages such as *for kids*, *natural* or *no preservatives* are checked against colours, flavourings, preservatives, sugar and the score.

Algorithm 5 - Alternatives ranking

Candidates in the same category are scored with Algorithm 3 and evaluated with Algorithm 2 for the selected members; any candidate that is AVOID for a member or does not beat the scanned score is removed. The rank value is score + feedback adjustment - 3 x (members with CAUTION). Feedback adds +8 per thumbs-up and -8 per thumbs-down on the same product, -10 for a disliked brand, and a sugar-proportional penalty after 'too sweet' feedback. Swap impact = (base - candidate nutrient per 100 g) x grams logged this week (or one serving a day) / 100.

3. Implementation

3.1 Hardware and software requirements

Category	Requirement
Hardware (development / demo)	Any laptop with 4 GB RAM or more, dual-core CPU, 1 GB free disk; internet for Gemini and Open Food Facts. Optional: smartphone on the same Wi-Fi for camera capture.
Operating system	Windows 10/11, macOS or Linux
Backend	Python 3.11+, FastAPI 0.143, Uvicorn, Pydantic v2, google-genai 2.31 (Gemini SDK), Pillow, Requests, Pandas, SQLite 3 (built in), pytest
Frontend	Node.js 18+, React 18.3, Vite 5.4, Tailwind CSS 3.4, React Router 6, Recharts 2.15
External services	Google Gemini API (gemini-3.6-flash primary; gemini-3.5-flash-lite and gemini-3.8-flash fallback); Open Food Facts API v2 (product and search)
Browser	Any modern browser; clipboard paste and text-to-speech use standard Web APIs

3.2 Supporting resources

Resource	Contents
sugar_aliases.csv	54 sugar names incl. jaggery, gur, khandsari, invert syrup, maltodextrin, with group and note
additives.csv	49 INS/E-number additives with class, plain-English purpose, concern level and note
allergens.json	22 allergen groups with Indian keywords (kaju, til, besan), parent groups and exclusion phrases
diet_rules.csv	34 rules marking items avoid/check for vegetarian, vegan, Jain and halal diets
indian_products_fallback.csv	73 common Indian packaged products (approximate values) across 17 categories
demo_labels.json	4 complete demo labels (incl. front-of-pack data) that work offline
Database (SQLite)	household, scans, extraction_cache, intake_logs, alt_feedback, shopping_list, llm_cache, off_cache, llm_calls

The REST API exposes about 30 endpoints, documented automatically by FastAPI at */docs*, including */api/scan*, */api/scan/barcode*, */api/scan/explain*, */api/alternatives/{id}*, */api/compare*, */api/log*, */api/tracker*, */api/report*, */api/products* and */api/lookup*. All errors return a consistent *{error, detail}* JSON body that the interface shows as a friendly message.

3.3 Sample code

(a) Deterministic health score (backend/rules/scoring.py, excerpt):

```
sugar = n.get("sugar_g")
if sugar is not None:
    add("sugar", "Sugar", sugar, -min(35.0, sugar * (3.5 if liquid else 0.875)), f"{sugar} g per 100g")
sodium = n.get("sodium_mg")
if sodium is not None:
    add("sodium", "Sodium", sodium, -min(25.0, sodium * 0.025), f"{sodium} mg per 100g")
...
weights = {"high": 5.0, "medium": 2.0, "low": 0.5}
add_pen = min(15.0, sum(weights.get(a.get("concern"), 0.0) for a in additives))
level = processing_level(ingredients, additives)          # 1, 3 or 4
add("processing", "Processing level", level, -{1: 0.0, 3: 5.0, 4: 10.0}[level], ...)
raw = round(100 + sum(c["points"] for c in comps), 1)      # avoid float drift
score = int(max(0, min(100, math.floor(raw + 0.5))))
```

(b) Allergy rule for one family member (backend/rules/engine.py, excerpt):

```
for raw in m.allergens:
    key = data.resolve_allergen(raw)                # "kaju" / "cashews" -> "cashew"
    a = present.get(key)
    if a and a["source"] == "ingredients":
        flag(AVOID, "allergen", f"Contains {a['display']} (listed allergy)", evidence(a))
    elif a:
        flag(WARN, "allergen", f"May contain traces of {a['display']}", evidence(a))
    elif table[key].get("parent") in present:       # generic "nuts" for a cashew allergy
        flag(WARN, "allergen", "Contains tree nuts that may include cashew - check the pack", ...)
```

(c) Gemini call with automatic model fallback (backend/llm/gemini_client.py, excerpt):

```
models = [config.GEMINI_MODEL] + [m for m in config.GEMINI_FALLBACK_MODELS if m != config.GEMINI_MODEL]
healthy = [m for m in models if self._down_until.get(m, 0) <= time.monotonic()] or models
for model in healthy:
    try:
        return self._call(client, model, purpose, contents, temperature, max_tokens, timeout_s)
    except _ModelUnavailable as exc:                # 429 / 5xx / timeout
        self._down_until[model] = time.monotonic() + config.GEMINI_MODEL_COOLDOWN_S
        last = LLMError(str(exc), exc.kind)
raise last
```

(d) Front-of-pack check for a pictured food (backend/rules/front_back.py, excerpt):

```
pos, ing, pct = _find(ingredients, item)            # position, text and % in the ingredient list
named = contains_term(norm(ex.product_name), _singular(item))
if pct is not None and pct < 5:
    verdict, reason = "misleading", f"Pictured on the front, but only {pct:g}% of the product."
elif named and pct is not None and pct < 15:
    verdict, reason = "misleading", f"The product is named after {item}, but it is only {pct:g}% ..."
```

3.4 Snapshots

The snapshots below were captured from the running application with a sample household of four members (a Jain, pre-diabetic adult allergic to cashew; a lactose-intolerant 9-year-old; a mother with hypertension; and an uncle with diabetes and heart disease).

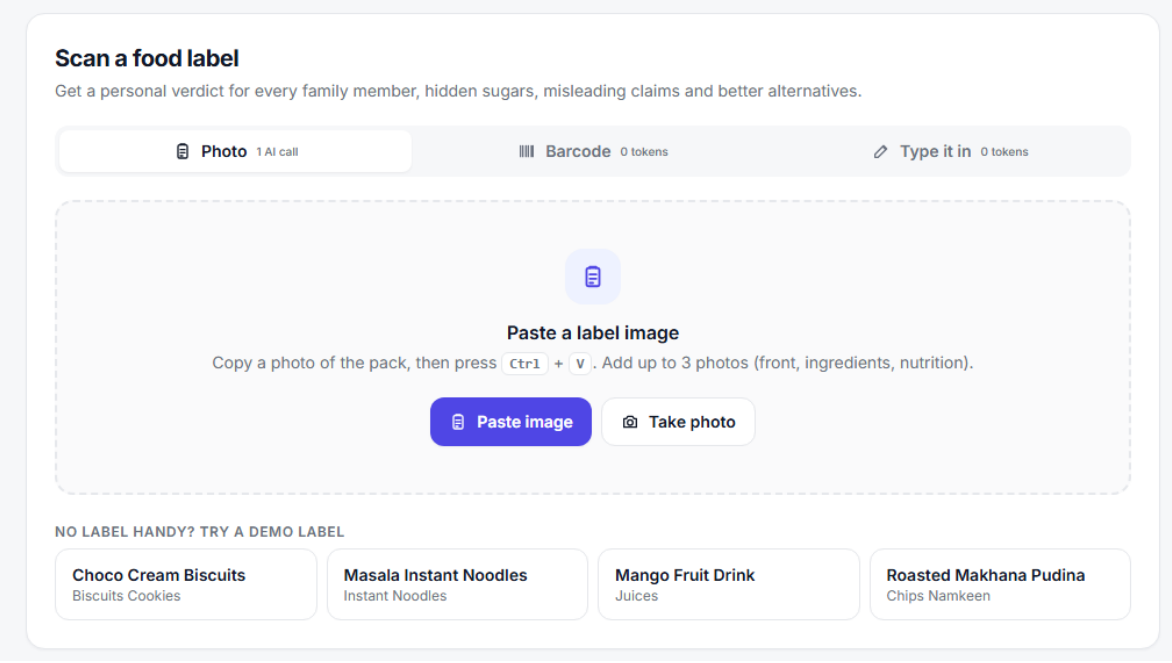


Figure 2: Scan page - paste or photograph up to three images, barcode lookup, manual entry and demo labels.

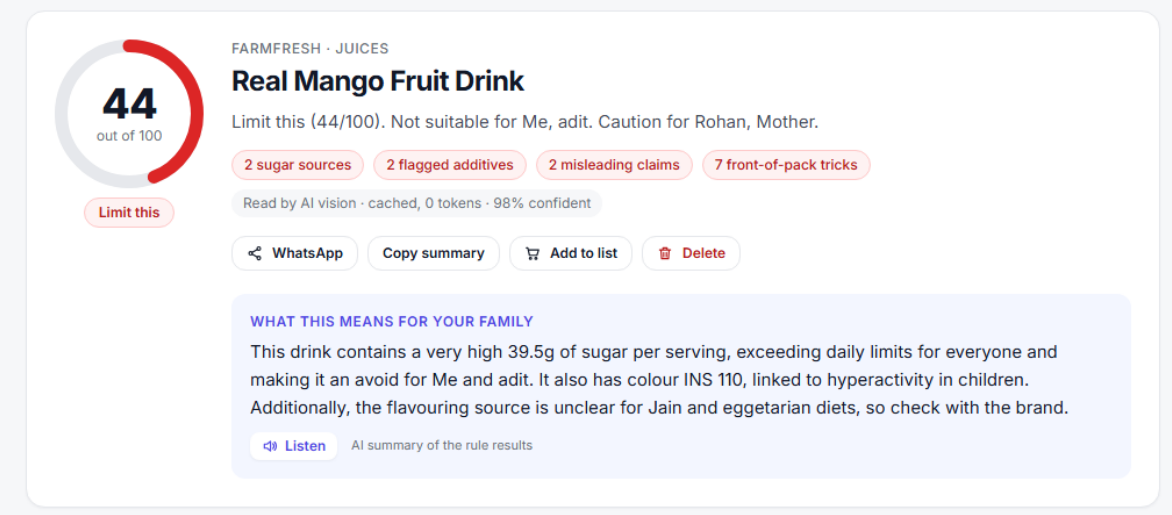


Figure 3: Result summary - score, verdict, problem counts, actions and the AI-written family summary with text-to-speech.

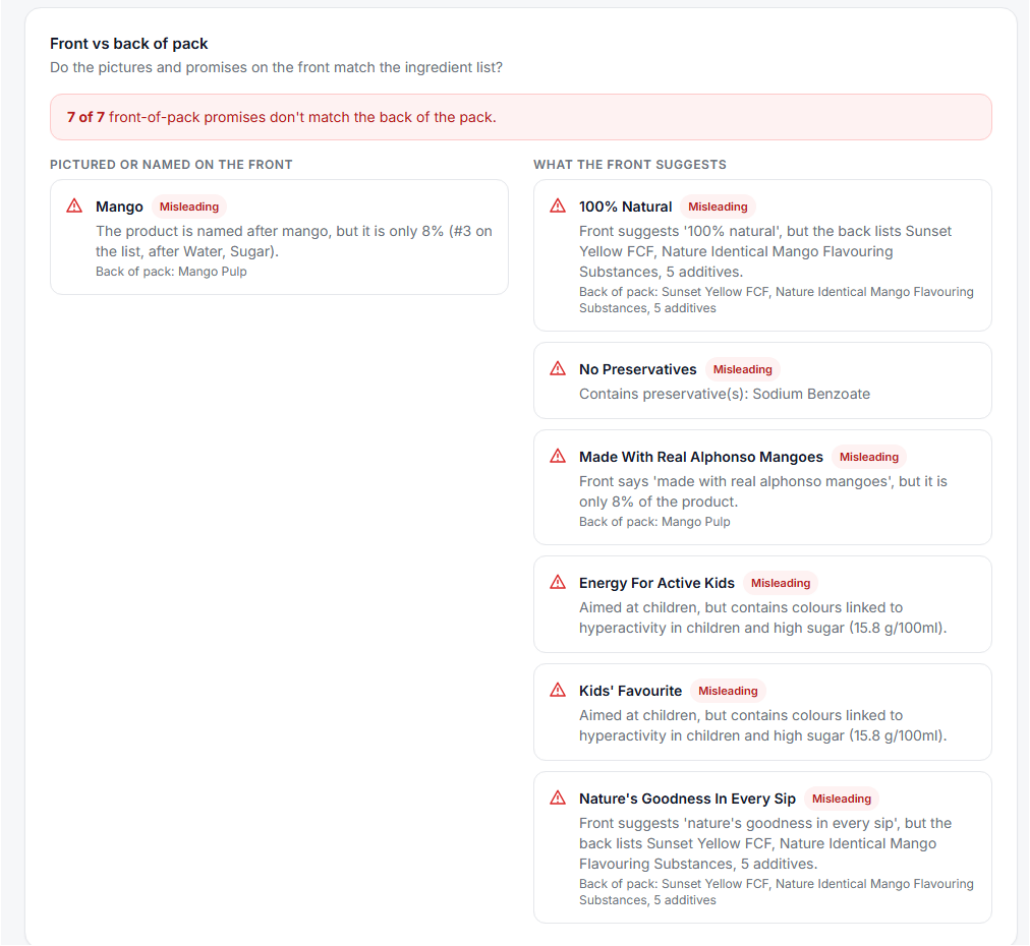


Figure 4: Front vs back of pack - all seven front-of-pack promises of a sample mango drink contradicted by its ingredient list.

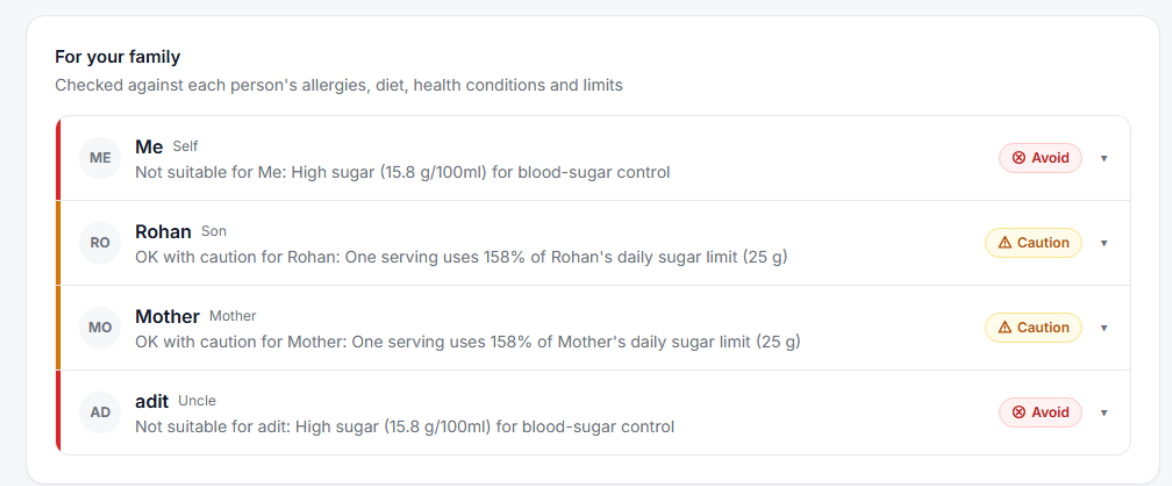


Figure 5: Per-member verdicts - the same product is AVOID for two members and CAUTION for two others.

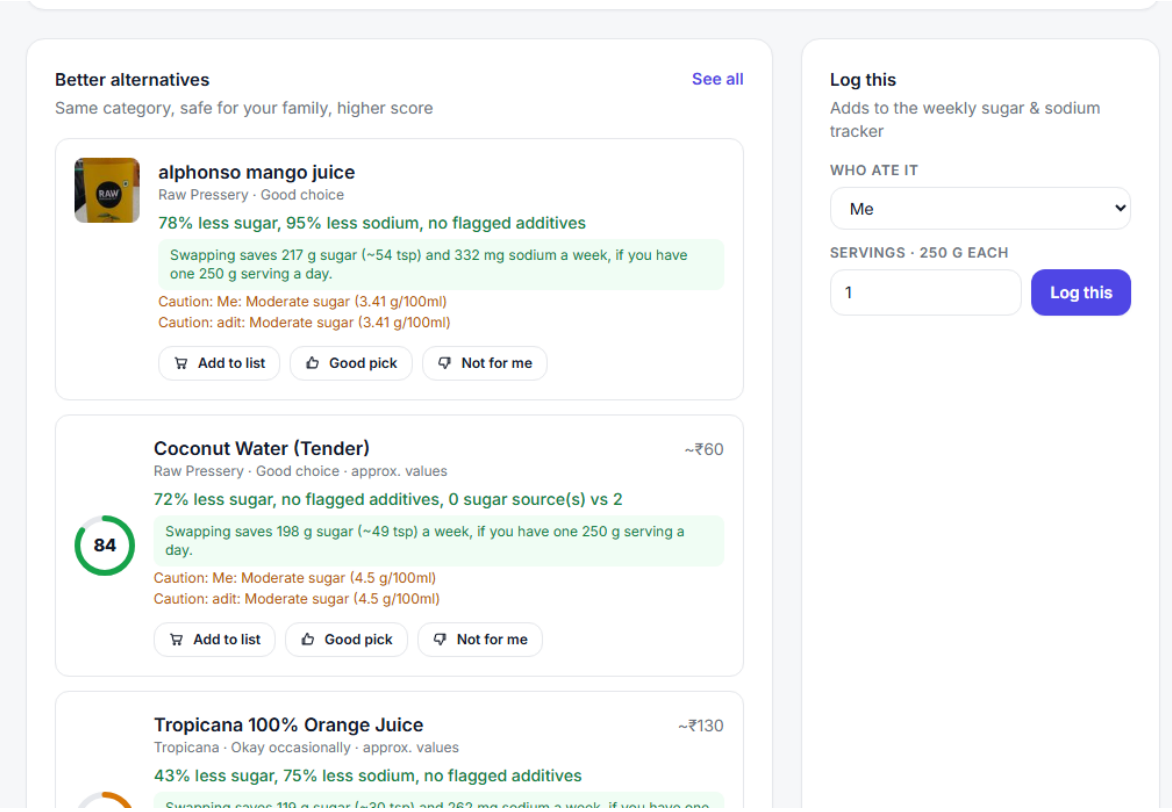


Figure 6: Alternatives with reasons, swap impact, shopping list and feedback, beside the intake logger.

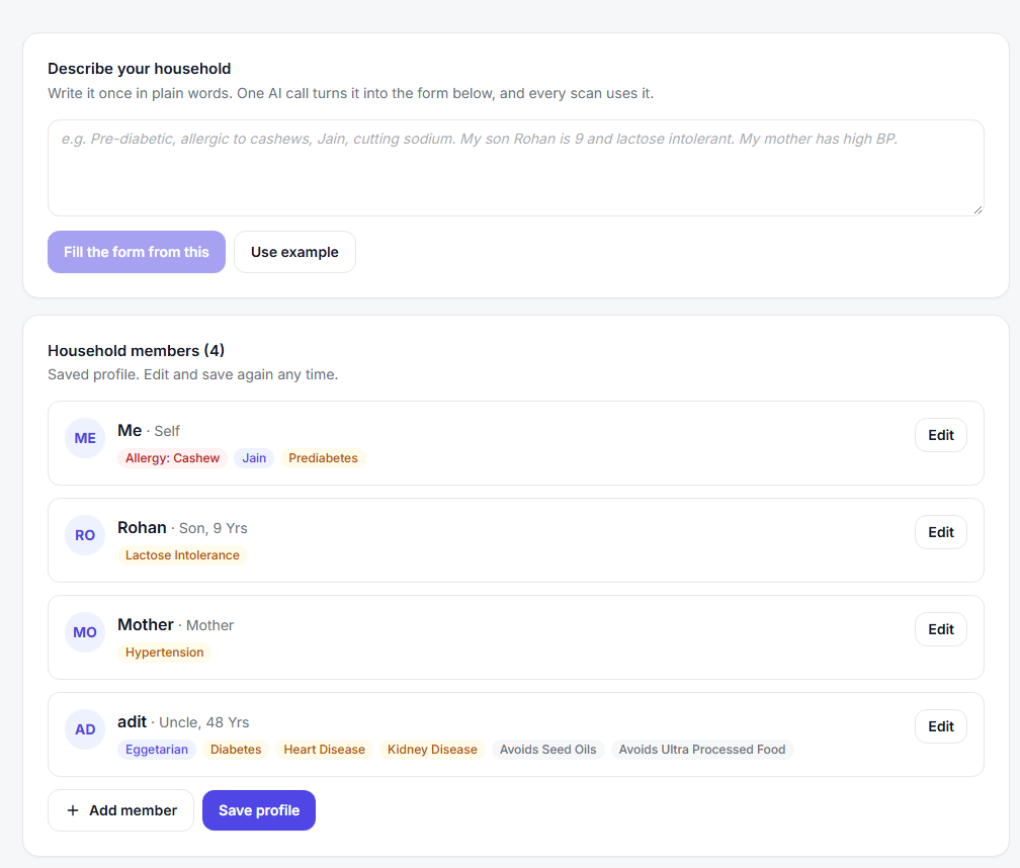


Figure 7: Family profile - free-text parsing into editable member cards.

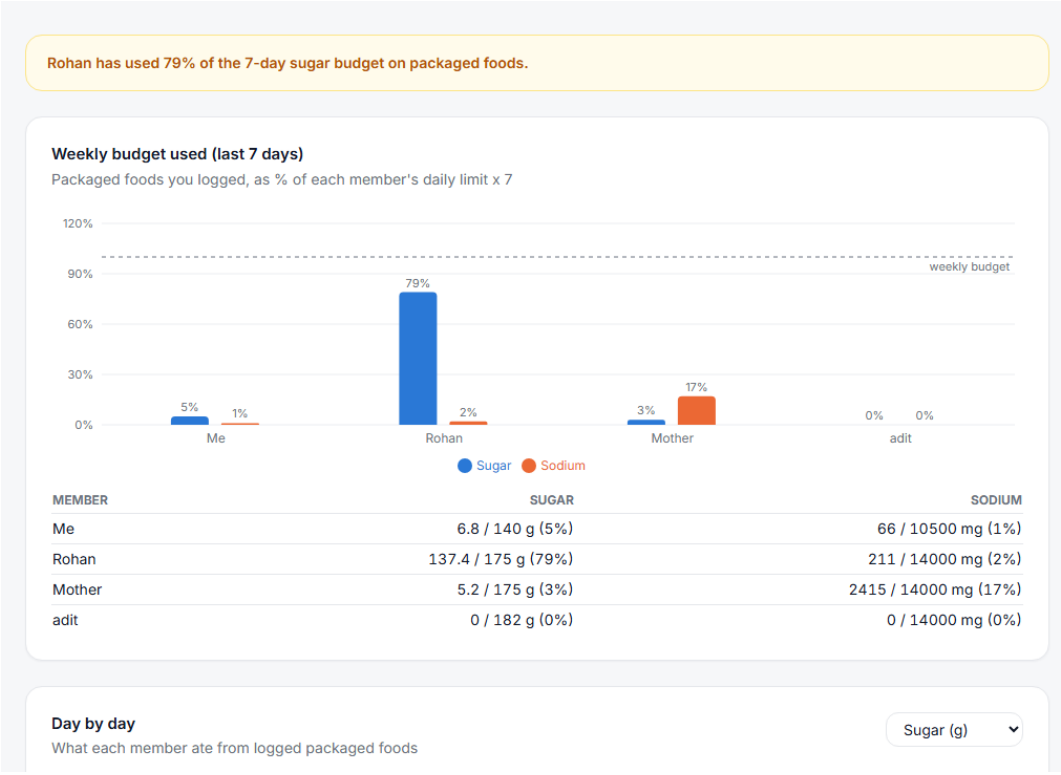


Figure 8: Tracker - weekly sugar and sodium budget used by each member (daily limit x 7).

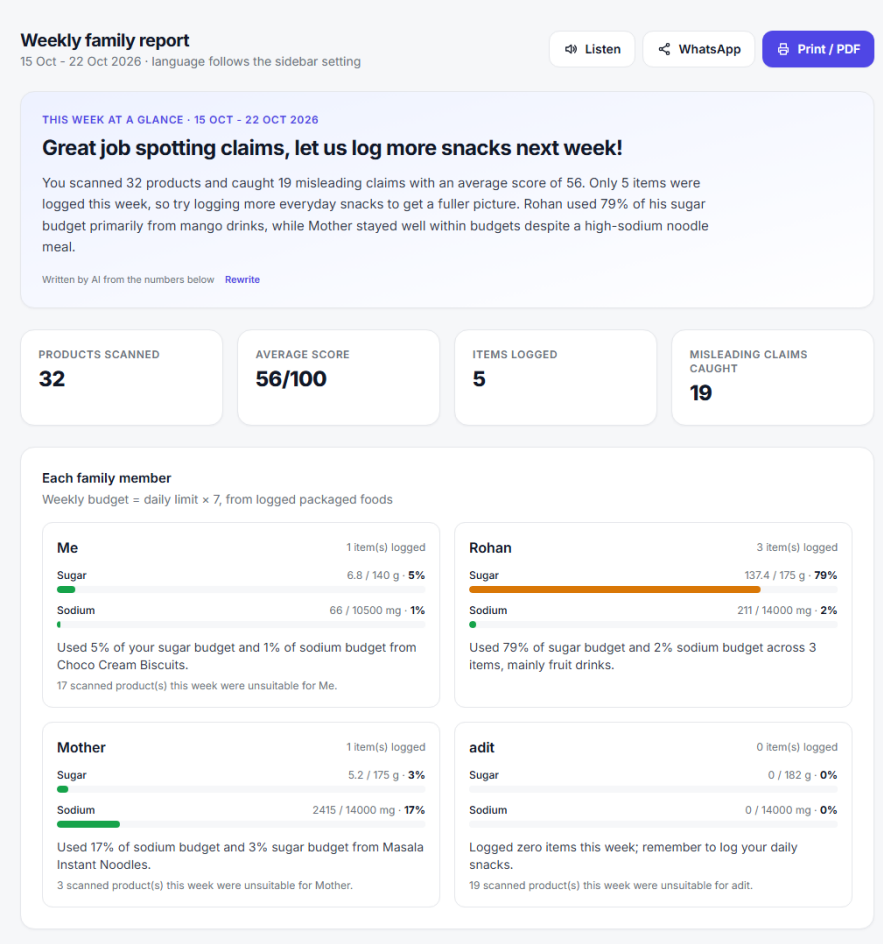


Figure 9: Weekly family report - numbers computed by the app, write-up generated by the LLM.

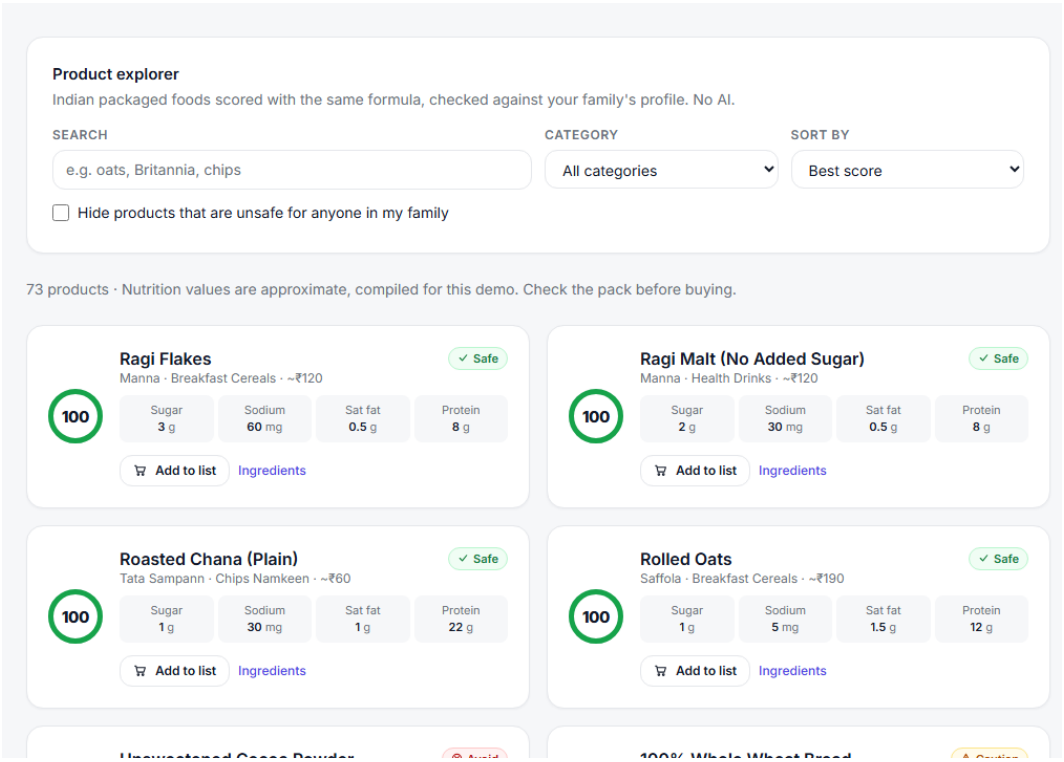


Figure 10: Product explorer - Indian products scored with the same formula and checked for the family.

4. Conclusion

4.1 Overall efficiency

The figures below were measured on the running system (Gemini usage is logged per call in the *llm_calls* table).

Measure	Result
Label extraction (new image set)	About 2,100 input + 450 output tokens; average 4.8 s (range 1.7-12.5 s) for successful calls
Repeat scan of the same photo / barcode lookup	0 tokens (cache / Open Food Facts); under 1 s
Family summary	About 340 input + 175 output tokens per scan and language; cached afterwards
Weekly report write-up	About 900 input + 360 output tokens; 3.3 s; cached repeat 0.7 s
Analysis modules (flags, score, claims, front/back, alternatives, tracker)	0 tokens; computed in milliseconds
Determinism	Same extraction always yields an identical analysis and score (automated test, 5 runs)
Automated tests	46 of 46 passing (rules, scoring, claims, API, caching, error handling)
Profile parser baseline	Keyword fallback reaches 87% field accuracy (52/60) on 10 sample profiles
Front-vs-back detection (sample packs)	7 of 7 misleading promises on a mango drink and 5 of 5 on a cereal; 0 false alarms on an honest makhana snack
Availability	During testing the newest Gemini model frequently timed out (504/503); the automatic fallback still completed scans, and barcode, manual and demo modes work without the LLM

4.2 Discussion

LabelGuard shows that splitting the problem between a multimodal LLM and a deterministic rules engine gives the best of both: the model reliably reads messy photographs (including front-of-pack imagery) in a single low-cost call, while every verdict, score and recommendation is reproducible, explainable with evidence and free to recompute when the family profile changes. Caching and the multi-model fallback keep the application responsive and inexpensive; the per-member verdicts, Indian-specific knowledge and front-versus-back analysis provide value that a general chatbot does not.

4.3 Limitations and future work

- Extraction quality depends on the photo; low light, curved packs or tiny fonts can lead to missed values (shown to the user as confidence and unreadable fields).
- Ingredient matching is keyword based; unusual spellings and regional-language labels need more synonyms or translation.
- The local product table uses approximate values; a verified brand database would be required for production.
- The score is a transparent heuristic, not a clinically validated index; a dietitian review of thresholds is planned.
- Future work: live barcode scanning with the camera, a fully translated Hindi/Tamil interface, user accounts, a larger labelled evaluation set and a study with real families.

5. References

[1] Food Safety and Standards Authority of India (FSSAI). *Food Safety and Standards (Labelling and Display) Regulations*, 2020.

[2] Food Safety and Standards Authority of India (FSSAI). *Food Safety and Standards (Advertising and Claims) Regulations*, 2018.

[3] UK Department of Health and Food Standards Agency. *Guide to Creating a Front of Pack (FoP) Nutrition Label for Pre-packed Products Sold through Retail Outlets*, 2016.

[4] World Health Organization. *Guideline: Sugars Intake for Adults and Children*. Geneva: WHO, 2015.

[5] World Health Organization. *Guideline: Sodium Intake for Adults and Children*. Geneva: WHO, 2012.

[6] Indian Council of Medical Research - National Institute of Nutrition. *Dietary Guidelines for Indians*, 2024.

[7] Monteiro, C. A., et al. Ultra-processed foods: what they are and how to identify them. *Public Health Nutrition*, 22(5), 936-941, 2019.

[8] McCann, D., et al. Food additives and hyperactive behaviour in 3-year-old and 8/9-year-old children in the community. *The Lancet*, 370(9598), 1560-1567, 2007.

[9] Codex Alimentarius Commission. *Class Names and the International Numbering System for Food Additives (CXG 36-1989)*, updated.

[10] Open Food Facts. *Open Food Facts API documentation (v2)*. https://openfoodfacts.github.io/openfoodfacts-server/api/

[11] Google. *Gemini API documentation: models, structured output and thinking*. https://ai.google.dev/gemini-api/docs

[12] FastAPI documentation. https://fastapi.tiangolo.com/ ; React documentation. https://react.dev/ ; Recharts. https://recharts.org/